Hoja de fórmulas: Django ORM con el proyecto Butaca Cero

Guía para pasar de SQL a Django usando tus propios modelos. Todos los ejemplos usan los nombres de campo de Python (`title`), nunca el `db_column` (`titulo`).

0. Mapa de modelos

Modelo	Campos clave	Relaciones
Customer, Usher, Manager, Operator	`user`	OneToOne a User. Accesos inversos: `user.customer_profile` , `usher_profile` , `manager_profile` , `operator_profile` . Customer, Usher y Manager tienen PK propia (`customer_id` , etc.)
Movie	title, synopsis, genre, rating, duration_minutes, image	`movie.screenings` (inversa de Screening)
Room	name, layout_configuration (JSON)	`room.seats` , `room.screenings` . `capacity` es una property, no una columna
SeatType	name, base_price, color, icon	`seat_type.seats` , `seat_type.promotions`
Seat	room, row, column, seat_type	FK a Room (CASCADE) y SeatType (PROTECT). Único por (room, row, column)
Screening	movie, room, starts_at, status, ticket_price, prices_by_type, room_layout_snapshot, capacity_snapshot	FK a Movie y Room (PROTECT). Tiene máquina de estados
Promotion	name, is_active, discount_type, discount_value, weekdays, starts_at, ends_at	ManyToMany con SeatType

Modelo	Campos clave	Relaciones
CinemaSettings	name, slogan, seat_reservation_minutes, etc.	Funciona como singleton: `CinemaSettings.current()`
Juego	GameAccount, Question, WheelConfig, WheelSlot, TriviaRun, TriviaAnswer, QuestionUse, WheelSpin, PrizeCoupon, GameEvent	FKs a User, `candy.CandyProduct`, `candy.CandyOrder`

Idea central: cada modelo es una tabla, cada campo una columna, cada instancia una fila. `Modelo.objects` es el gestor (manager) por donde pasan todas las consultas.

1. Imports y shell

```
# python manage.py shell
from decimal import Decimal
from datetime import timedelta
from django.db import transaction
from django.db.models import Q, F, Count, Sum, Avg, Min, Max, Exists,
OuterRef
from django.utils import timezone
from domain.movies.models import Movie
from domain.rooms.models import Room
from domain.seat_types.models import SeatType
from domain.screenings.models import Screening
from domain.promotions.models import Promotion
# Seat: importalo desde la app donde esté (ajustá la ruta)
```

2. Tabla de equivalencias SQL ↔ ORM

SQL	Django
`SELECT * FROM movies`	`Movie.objects.all()`
`WHERE a = 1 AND b = 2`	`.filter(a=1, b=2)`
`WHERE NOT (a = 1)`	`.exclude(a=1)`

SQL	Django
Una sola fila	`.get(pk=1)` (lanza `DoesNotExist` o `MultipleObjectsReturned`)
`ORDER BY x DESC`	`.order_by('-x')`
`LIMIT 5` / `OFFSET 5`	`[:5]` / `[5:10]`
`COUNT(*)`	`.count()`
`EXISTS`	`.exists()`
`SELECT a, b`	`.values('a', 'b')` o `.values_list('a', flat=True)`
`DISTINCT`	`.distinct()`
`LIKE '%x%'`	`.filter(campo__icontains='x')`
`IN (1,2)`	`.filter(campo__in=[1, 2])`
`BETWEEN`	`.filter(campo__range=(a, b))`
`IS NULL`	`.filter(campo__isnull=True)`
`OR`	`Q(a=1) | Q(b=2)` (ver sección 6)
`JOIN`	doble guion bajo: `.filter(movie__genre='TERROR')`
`GROUP BY` + `COUNT`	`.values('x').annotate(n=Count('id'))`
`HAVING`	`.filter(n__gt=3)` después del `annotate`
`INSERT`	`.create(...)` o `obj.save()`
`UPDATE`	`obj.save()` o `queryset.update(...)`
`DELETE`	`obj.delete()` o `queryset.delete()`

3. CREATE (INSERT)

```
movie = Movie.objects.create(
    title='Dune', synopsis='...', genre=Movie.Genre.CIENCIA_FICCION,
    rating=Movie.Rating.MAS_13, duration_minutes=155,
image='peliculas/dune.jpg',
```

```
)

# Equivalente en dos pasos (útil si querés modificar antes de guardar)
movie = Movie(title='Dune', ...)
movie.save()

# Claves foráneas: pasá el objeto o el _id
room = Room.objects.create(name='Sala 1')
vip = SeatType.objects.create(name='VIP', base_price=Decimal('12.50'),
color='#FFD700')
Seat.objects.create(room=room, row=0, column=0, seat_type=vip)
Seat.objects.create(room_id=room.pk, row=0, column=1, seat_type_id=vip.pk)

# Muchas filas de una (1 solo INSERT)
Seat.objects.bulk_create([
    Seat(room=room, row=r, column=c, seat_type=vip)
    for r in range(5) for c in range(8)
])

# Perfil 1 a 1 con usuario
Customer.objects.create(user=request.user)

# Many-to-Many: NO se pasa en create(), se agrega después
promo = Promotion.objects.create(name='Miércoles',
discount_type='percentage', discount_value=Decimal('20'))
promo.seat_types.add(vip)

# Función: starts_at debe ser aware (con zona horaria) y ticket_price es
obligatorio
scr = Screening.objects.create(
    movie=movie, room=room,
    starts_at=timezone.now() + timedelta(days=2),
    ticket_price=Decimal('8.00'),
)
# Screening.save() está sobrescrito: al ser capacity_snapshot == 0 llama
# a capture_room_configuration() y llena snapshot, precios y capacidad solo.
```

Ojo:

- `create()` y `save()` NO ejecutan `clean()` ni los validators (ej. `MinValueValidator`). Para validar, llamá `obj.full_clean()` antes de guardar.
- `bulk_create` no llama a `save()` ni a señales, así que se saltea lógica como la de `Screening.save()`.

get_or_create / update_or_create

```
# Devuelve (objeto, creado_bool). Evita duplicados sin race condition simple.
acc, created = GameAccount.objects.get_or_create(user=request.user, defaults=
{'spins': 1})

obj, created = SeatType.objects.update_or_create(
    name='VIP', defaults={'base_price': Decimal('15.00'), 'color':
'#FFD700'},
)
```

4. READ (SELECT)

```
Movie.objects.all()                          # todas (QuerySet, lazy: no
consulta hasta que lo iterás)
Movie.objects.get(pk=3)                      # exactamente una
Movie.objects.filter(genre='TERROR')         # varias
Movie.objects.exclude(rating='+18')          # NOT
Movie.objects.filter(genre='TERROR').first() # primera o None
Movie.objects.order_by('-duration_minutes').first()
Movie.objects.last()                         # última según ordering
Movie.objects.filter(title='x').exists()     # True/False (barato)
Movie.objects.count()

# Slicing = LIMIT/OFFSET
Movie.objects.order_by('title')[:10]
Movie.objects.order_by('title')[10:20]

# Solo algunas columnas
Movie.objects.values('id', 'title')                     # lista de dicts
Movie.objects.values_list('title', flat=True)           # lista de strings
Movie.objects.values_list('id', 'title')                # lista de tuplas
Movie.objects.values_list('genre', flat=True).distinct() # géneros distintos

# in_bulk: diccionario {pk: objeto}
Movie.objects.in_bulk([1, 2, 3])

# Encadenar es lo normal: cada método devuelve un nuevo QuerySet
Screening.objects.filter(status='publicada').filter(starts_at__gt=timezone.no
w()).order_by('starts_at')
```

get() seguro:

```
from django.shortcuts import get_object_or_404
movie = get_object_or_404(Movie, pk=pk)   # en vistas: devuelve 404 si no
existe

try:
    movie = Movie.objects.get(pk=pk)
except Movie.DoesNotExist:
    movie = None
```

PK distinta de `id`: Customer, Usher y Manager definen su propia PK (`customer_id`). Usá siempre `pk=` para ser agnóstico: `Customer.objects.get(pk=1)`. En Operator la PK sigue siendo `id`.

5. Lookups (los sufijos con doble guion bajo)

Formato: `campo__lookup=valor`.

```
# Texto
Movie.objects.filter(title__icontains='dun')      # LIKE %dun% sin distinguir
mayúsculas
Movie.objects.filter(title__istartswith='the')
Movie.objects.filter(title__iexact='dune')
# (sin i = distingue mayúsculas: contains, startswith, endswith, exact)

# Números y fechas
Movie.objects.filter(duration_minutes__gte=120)   # >=   (gt, gte, lt, lte)
Movie.objects.filter(duration_minutes__range=(90, 150))
Screening.objects.filter(starts_at__gt=timezone.now())
Screening.objects.filter(starts_at__date=timezone.localdate())      # solo la
fecha
Screening.objects.filter(starts_at__year=2026, starts_at__month=11)
Screening.objects.filter(starts_at__week_day=6)   # 1=domingo ... 7=sábado
(OJO: distinto a weekday() de Python)

# Listas y nulos
Movie.objects.filter(genre__in=['TERROR', 'SUSPENSO'])
Promotion.objects.filter(starts_at__isnull=False)  # tiene horario

# Choices con TextChoices: usá la constante, no el string a mano
Screening.objects.filter(status=Screening.Status.PUBLISHED)
```

```
# JSONField (según el motor de BD)
Screening.objects.filter(prices_by_type__has_key='3')   # existe esa clave
Promotion.objects.filter(weekdays__contains=[2])        # PostgreSQL; en
SQLite filtrá en Python
```

6. Q (OR / NOT) y F (comparar columnas)

```
# OR
Movie.objects.filter(Q(genre='TERROR') | Q(genre='SUSPENSO'))
# AND explícito y NOT
Movie.objects.filter(Q(duration_minutes__gt=120) & ~Q(rating='+18'))

# Combinación típica de búsqueda
q = 'dune'
Movie.objects.filter(Q(title__icontains=q) | Q(synopsis__icontains=q))

# Construcción dinámica de filtros (como un buscador con campos opcionales)
filters = Q()
if genre:
    filters &= Q(genre=genre)
if min_duration:
    filters &= Q(duration_minutes__gte=min_duration)
Movie.objects.filter(filters)

# F(): referirse a otra columna (o a la misma) dentro de la consulta
Screening.objects.filter(ticket_price__gt=F('room__seats__seat_type__base_pri
ce'))
SeatType.objects.update(base_price=F('base_price') * Decimal('1.10'))   #
+10% sin traer filas a Python
```

7. Relaciones (los JOIN de Django)

Hacia adelante (FK, OneToOne)

```
scr = Screening.objects.get(pk=1)
scr.movie.title          # JOIN implícito (1 consulta extra si no hay
select_related)
scr.room.name
scr.movie_id             # el valor crudo de la FK, sin consulta
```

Hacia atrás (por `related_name`)

```
movie.screenings.all()                # funciones de la película
room.seats.count()                    # asientos de la sala
vip.seats.filter(room=room)
vip.promotions.all()                  # promociones de ese tipo (M2M
inversa)
user.customer_profile                 # OneToOne inversa: lanza
RelatedObjectDoesNotExist si no hay
hasattr(user, 'customer_profile')     # forma segura de preguntar
```

Si la FK no tiene `related_name` (como `TriviaRun.user`), el acceso inverso es `user.triviarun_set` y en filtros `triviarun__...`.

Filtrar atravesando relaciones

```
# JOIN en la consulta: lo que sigue al __ es un campo del modelo relacionado
Screening.objects.filter(movie__genre='TERROR', room__name='Sala 1')
Seat.objects.filter(seat_type__name='VIP', room__name='Sala 1')

# Hacia atrás en filtros: usa el related_name (o el nombre del modelo en
minúscula)
Movie.objects.filter(screenings__status='publicada').distinct()    #
películas con alguna función publicada
Room.objects.filter(seats__seat_type__name='VIP').distinct()

# Películas SIN funciones
Movie.objects.filter(screenings__isnull=True)
Movie.objects.exclude(screenings__isnull=False)
```

Usá `.distinct()` cuando el JOIN pueda duplicar filas (relaciones 1 a N).

Many-to-Many

```
promo.seat_types.add(vip, standard)   # agrega
promo.seat_types.remove(vip)          # quita
promo.seat_types.set([vip])           # reemplaza todo
promo.seat_types.clear()              # vacía
promo.seat_types.all()
promo.seat_types.filter(name='VIP').exists()
Promotion.objects.filter(seat_types=vip)
```

Evitar N+1 consultas

```
# select_related: JOIN en una sola consulta (para FK y OneToOne, o sea hacia
UNO)
Screening.objects.select_related('movie', 'room')

# prefetch_related: segunda consulta + unión en Python (para inversas y M2M,
o sea hacia MUCHOS)
Promotion.objects.prefetch_related('seat_types')
Room.objects.prefetch_related('seats__seat_type')
```

Ejemplo real de tu código: `Room.__str__` usa `self.capacity`, que hace `self.seats.count()` y por lo tanto una consulta por sala. Un listado de 50 salas hace 51 consultas. Se evita anotando con `Room.objects.annotate(_capacity=Count('seats'))` (por eso la property busca un atributo `_capacity` primero).

8. Agregaciones y GROUP BY

```
# aggregate: UNA fila de resultado, devuelve dict
SeatType.objects.aggregate(Avg('base_price'), Max('base_price'))
# {'base_price__avg': ..., 'base_price__max': ...}
SeatType.objects.aggregate(precio_max=Max('base_price'))   # con alias
Movie.objects.aggregate(total=Sum('duration_minutes'))

# annotate: agrega una columna calculada a CADA fila
Room.objects.annotate(total_asientos=Count('seats'))
Movie.objects.annotate(n_funciones=Count('screenings'))

# HAVING = filter DESPUÉS de annotate
Movie.objects.annotate(n=Count('screenings')).filter(n__gte=3)
```

```
# Filtrar lo que se cuenta (COUNT condicional)
Movie.objects.annotate(
    publicadas=Count('screenings', filter=Q(screenings__status='publicada'))
)

# GROUP BY: values() ANTES de annotate
Movie.objects.values('genre').annotate(total=Count('id')).order_by('-total')
# -> [{'genre': 'TERROR', 'total': 4}, ...]

# Cantidad de asientos por tipo en una sala
Seat.objects.filter(room=room).values('seat_type__name').annotate(n=Count('id
'))

# Exists / OuterRef: subconsulta correlacionada
futuras = Screening.objects.filter(movie=OuterRef('pk'),
starts_at__gt=timezone.now())
Movie.objects.annotate(tiene_futuras=Exists(futuras)).filter(tiene_futuras=Tr
ue)
```

Regla mental: `aggregate` colapsa todo a un valor. `annotate` calcula por fila. `values().annotate()` agrupa.

Límite: solo se puede filtrar, ordenar o agregar por columnas reales o anotaciones, no por `@property`. `Room.capacity` y `Screening.ends_at` son properties de Python. Para filtrar por capacidad: `Room.objects.annotate(cap=Count('seats')).filter(cap__gte=50)`.

9. UPDATE

```
# Opción 1: traer, modificar, guardar. Corre save() y la lógica del modelo.
movie = Movie.objects.get(pk=1)
movie.title = 'Nuevo título'
movie.save()
movie.save(update_fields=['title'])   # solo esa columna (más seguro ante
concurrencia)

# Opción 2: UPDATE masivo en una consulta. NO llama save() ni señales.
Movie.objects.filter(rating='+18').update(rating='+16')
Promotion.objects.filter(is_active=True).update(is_active=False)
Screening.objects.filter(pk__in=ids).update(status=Screening.Status.IN_PROGRE
SS)
n = Movie.objects.filter(...).update(...)   # devuelve la cantidad de filas
afectadas

# Con F: modifica en la BD, evita race conditions (leer-modificar-escribir)
GameAccount.objects.filter(user=user).update(spins=F('spins') + 1)

# Muchos objetos con valores distintos
bulk_update(objs, ['campo1', 'campo2'])   # ej:
Seat.objects.bulk_update(seats, ['seat_type'])
```

Cuándo usar cuál:

- Si el modelo tiene lógica en `save()`, usá `save()`. Por ejemplo, pasar una función a CANCELED dispara reembolsos desde `Screening.save()`. Con `.update(status='cancelada')` ese código no corre.
- Tu `Screening.finalize_expired()` usa `.update()` a propósito, para cambiar muchas funciones a FINISHED sin disparar `save()`.

10. DELETE

```
movie.delete()
Movie.objects.filter(genre='TERROR').delete()   # devuelve (total,
{'app.Modelo': n, ...})
```

Qué pasa con las filas relacionadas lo define `on_delete`:

on_delete	Comportamiento	Ejemplo en tu código
`CASCADE`	Borra también las filas hijas	Borrar un Room borra sus Seat. Borrar un User borra su Customer
`PROTECT`	Impide borrar si hay hijas (lanza `ProtectedError`)	No se puede borrar una Movie con Screening, ni un SeatType usado por un Seat
`SET_NULL`	Pone la FK en NULL (requiere `null=True`)	`QuestionUse.question`, `GameEvent.user`

```
from django.db.models import ProtectedError
try:
    movie.delete()
except ProtectedError:
    # mostrar mensaje: la película tiene funciones
    ...
```

11. Transacciones y concurrencia

```
from django.db import transaction

# Todo o nada: si algo lanza excepción, se revierte todo
with transaction.atomic():
    scr = Screening.objects.create(...)
    Seat.objects.bulk_create([...])

# Decorador
@transaction.atomic
def comprar(...): ...

# select_for_update: bloquea la fila hasta terminar la transacción (evita
carreras)
with transaction.atomic():
    scr = Screening.objects.select_for_update().get(pk=pk)
    scr.change_status(Screening.Status.PUBLISHED)

# on_commit: ejecutar algo SOLO si la transacción se confirma (ej. enviar
mail)
transaction.on_commit(lambda: enviar_notificacion(ids))
```

`select_for_update` solo funciona dentro de `atomic()`. Tu `Screening.save()` y `change_status()` usan este patrón.

12. Validación y constraints

```
from django.core.exceptions import ValidationError
from django.db import IntegrityError

obj = Promotion(name='x', discount_type='percentage',
discount_value=Decimal('150'))
try:
    obj.full_clean()   # corre: validators de campos + clean_fields + clean()
+ unique
    obj.save()
except ValidationError as e:
    print(e.message_dict)   # {'discount_value': ['El descuento porcentual no
puede superar el 100%.']}
```

Dónde se valida:

- **Validators de campo** (`MinValueValidator`, `MaxValueValidator`): corren en `full_clean()` y en ModelForms, no en `save()`.
- **`clean()` del modelo**: validaciones entre campos. Se lanza `ValidationError({'campo': 'mensaje'})`.
- **Constraints** (`UniqueConstraint`, `CheckConstraint`, `unique=True`): las aplica la BD y fallan con `IntegrityError` al guardar.

```
# Seat tiene UniqueConstraint(room, row, column)
try:
    Seat.objects.create(room=room, row=0, column=0, seat_type=vip)   # ya
existía
except IntegrityError:
    ...

# Constraint condicional (TriviaRun): un solo run activo por usuario
# UniqueConstraint(fields=['user'], condition=Q(finished_at__isnull=True),
...)
```

13. Lógica de negocio dentro del modelo (patrones de tu código)

`@property`: valor calculado sin argumentos. Se usa como atributo (`movie.runtime_label`). No se puede usar en queries.

Método de instancia: lógica sobre un objeto (`promo.applies_to(seat_type_id, starts_at)`, `promo.discounted_price(base)`, `screening.price_for_type(type_id)`).

`@classmethod`: lógica sobre la tabla (`Screening.published()`, `CinemaSettings.current()`, `Screening.finalize_expired()`).

Máquina de estados de Screening:

```
scr.can_transition_to(Screening.Status.PUBLISHED)  # True/False según
TRANSITIONS
scr.available_transitions                          # estados permitidos ahora
scr.change_status(Screening.Status.PUBLISHED)      # valida, bloquea fila,
guarda snapshot
# Lanza ValidationError si la transición no es válida
```

Precio con promoción:

```
scr = Screening.objects.get(pk=1)
base = scr.price_for_type(vip.pk)
for promo in
Promotion.objects.filter(is_active=True).prefetch_related('seat_types'):
    if promo.applies_to(vip.pk, scr.starts_at):
        print(promo.name, promo.discounted_price(base))
```

Singleton de configuración:

```
config = CinemaSettings.current()
config.seat_reservation_minutes
```

14. Vistas CRUD mínimas (función)

```
from django.shortcuts import render, redirect, get_object_or_404
from django import forms

class MovieForm(forms.ModelForm):
    class Meta:
        model = Movie
        fields = ['title', 'synopsis', 'genre', 'rating', 'duration_minutes',
'image']

def movie_list(request):
    movies = Movie.objects.all()
    q = request.GET.get('q')
    if q:
        movies = movies.filter(title__icontains=q)
    return render(request, 'movies/list.html', {'movies': movies})

def movie_create(request):
    form = MovieForm(request.POST or None, request.FILES or None)   # FILES
por el ImageField
    if form.is_valid():          # corre validators y clean()
        form.save()
        return redirect('movie_list')
    return render(request, 'movies/form.html', {'form': form})

def movie_update(request, pk):
    movie = get_object_or_404(Movie, pk=pk)
    form = MovieForm(request.POST or None, request.FILES or None,
```

```
instance=movie)
    if form.is_valid():
        form.save()
        return redirect('movie_list')
    return render(request, 'movies/form.html', {'form': form})

def movie_delete(request, pk):
    movie = get_object_or_404(Movie, pk=pk)
    if request.method == 'POST':
        movie.delete()
        return redirect('movie_list')
    return render(request, 'movies/confirm_delete.html', {'movie': movie})
```

El `ModelForm` ya llama a `full_clean()` por vos, así que valida con los mismos `clean()` y validators del modelo. Para el admin: `admin.site.register(Movie)` o con `@admin.register(Movie)` y una clase `ModelAdmin` (`list_display`, `search_fields`, `list_filter`).

15. Recetas resueltas (tipo examen)

```
# 1. Funciones publicadas futuras, con película y sala, ordenadas
Screening.objects.filter(
    status=Screening.Status.PUBLISHED, starts_at__gt=timezone.now()
).select_related('movie', 'room').order_by('starts_at')

# 2. Funciones de hoy de una película
Screening.objects.filter(movie=movie, starts_at__date=timezone.localdate())

# 3. Cantidad de funciones por película, de mayor a menor
Movie.objects.annotate(n=Count('screenings')).order_by('-n')

# 4. Películas que nunca se proyectaron
Movie.objects.filter(screenings__isnull=True)

# 5. Salas con más de 50 asientos
Room.objects.annotate(cap=Count('seats')).filter(cap__gt=50)

# 6. Distribución de asientos por tipo en una sala
Seat.objects.filter(room=room).values('seat_type__name').annotate(n=Count('id
'))
```

```
# 7. Precio promedio por tipo de asiento
SeatType.objects.aggregate(promedio=Avg('base_price'))

# 8. Duración total y promedio de las películas de un género
Movie.objects.filter(genre='ACCION').aggregate(total=Sum('duration_minutes'),
prom=Avg('duration_minutes'))

# 9. Funciones que se superponen con un horario en una sala (lógica de
clean())
Screening.objects.filter(room=room).exclude(status='cancelada').exclude(pk=sc
r.pk)
# (ends_at es property: en tu clean() se compara en Python)

# 10. Promociones activas para el día de hoy
hoy = timezone.localdate().weekday()
[p for p in Promotion.objects.filter(is_active=True) if not p.weekdays or hoy
in p.weekdays]

# 11. Promociones de un tipo de asiento (o las que valen para todos)
Promotion.objects.filter(is_active=True).filter(Q(seat_types=vip) |
Q(seat_types__isnull=True)).distinct()

# 12. Clientes con su usuario en 1 consulta
Customer.objects.select_related('user')

# 13. Cancelar una función de forma segura
scr = Screening.objects.get(pk=pk)
if scr.can_transition_to(Screening.Status.CANCELED):
    scr.change_status(Screening.Status.CANCELED)

# 14. Cambiar el tipo de una fila entera de asientos
Seat.objects.filter(room=room, row=0).update(seat_type=vip)

# 15. Crear la grilla de una sala
Seat.objects.bulk_create([
    Seat(room=room, row=r, column=c, seat_type=standard)
    for r in range(rows) for c in range(cols)
])
```

16. Migraciones y comandos útiles

```
python manage.py makemigrations        # genera migración tras cambiar
models.py
python manage.py migrate               # aplica a la BD
python manage.py showmigrations
python manage.py sqlmigrate app 0001   # muestra el SQL que se va a ejecutar
python manage.py shell                 # consola para probar consultas
python manage.py createsuperuser
python manage.py dbshell               # consola SQL cruda
```

Para ver el SQL que genera una consulta: `print(Movie.objects.filter(genre='TERROR').query)`.

17. Errores típicos (y cómo evitarlos)

Error	Causa / solución
`DoesNotExist`	`get()` sin resultados. Usá `filter().first()` o `get_object_or_404`
`MultipleObjectsReturned`	`get()` encontró varios. Filtrá por algo único (`pk`)
`FieldError: Cannot resolve keyword`	Usaste un nombre equivocado, el `db_column` (`titulo`) o una property. Usá el nombre Python del campo
`IntegrityError`	Violaste unique/FK/null. Capturalo o validá antes
`ProtectedError`	Intentaste borrar algo referenciado con PROTECT
`RelatedObjectDoesNotExist`	Accediste a un OneToOne inverso inexistente. Usá `hasattr`
M2M en `create()` falla	Creá el objeto y después `.add()`
Naive datetime warning	Usá `timezone.now()`, no `datetime.now()`
Los valores `Decimal` se pierden	Usá `Decimal('10.50')` con string, nunca `Decimal(10.50)`
Resultados duplicados tras JOIN	Agregá `.distinct()`

Error	Causa / solución
`update()` no disparó mi lógica	`update()` no llama a `save()`. Usá `save()` por objeto si la lógica está ahí
Muchas consultas en un listado	Faltó `select_related` / `prefetch_related`
`select_for_update` falla	Tiene que estar dentro de `transaction.atomic()`

18. Checklist para resolver una feature en el examen

1. ¿Qué modelo(s) toca? Mirá el mapa de la sección 0 y los `related_name`.
2. ¿Es leer, crear, editar o borrar? Buscá el patrón en la sección correspondiente.
3. ¿Hay lógica ya hecha en el modelo (properties, métodos, `clean`, `save`)? Reutilizala en lugar de reescribirla.
4. ¿Necesita validación? `full_clean()` o `ModelForm`.
5. ¿Toca varias tablas a la vez? Envolvé en `transaction.atomic()`.
6. ¿Es un listado? Agregá `select_related` / `prefetch_related` y `order_by`.
7. Probá la consulta en `manage.py shell` antes de ponerla en la vista.